

Códigos-fonte gerados pelas LLMs

ChatGPT

```
# =====  
  
# ANÁLISE DE ESTABILIDADE E CUSTO  
# SISTEMA DE ABASTECIMENTO DE ÁGUA  
# =====  
  
import random  
  
import pandas as pd  
  
import matplotlib.pyplot as plt  
  
# =====  
  
# FUNÇÃO PARA CLASSIFICAR O RISCO  
# =====  
  
def classificar_risco(autonomia):  
  
    if autonomia > 3:  
        return "Baixo"  
  
    elif autonomia >= 1:  
        return "Médio"  
  
    else:  
        return "Alto"
```

```
# =====
```

```
# DADOS DOS BAIRROS
```

```
# =====
```

```
bairros = [
```

```
    "Centro",
```

```
    "Vila Nova",
```

```
    "Industrial",
```

```
    "Santa Rosa"
```

```
]
```

```
demandas = [1200, 900, 1500, 700]
```

```
capacidades = [5000, 3500, 6000, 3000]
```

```
volumes_iniciais = [4200, 2000, 2500, 2800]
```

```
perdas = [8, 12, 15, 5]
```

```
custos_bombeamento = [0.45, 0.50, 0.60, 0.40]
```

```
# =====
```

```
# LISTAS DE RESULTADOS
```

```
# =====
```

```
volumes_entregues = []
```

```
volumes_perdidos = []
```

```
custos_diarios = []
```

```
autonomias = []
```

```
riscos = []
```

```
# =====
```

```
# CÁLCULOS
```

```
# =====
```

```
for i in range(len(bairros)):
```

```
    # Volume perdido
```

```
    perdido = demandas[i] * (perdas[i] / 100)
```

```
    # Volume entregue
```

```
    entregue = demandas[i] - perdido
```

```
    # Custo diário
```

```
    custo = demandas[i] * custos_bombeamento[i]
```

```
    # Autonomia
```

```
autonomia = volumes_iniciais[i] / demandas[i]
```

```
# Classificação do risco
```

```
risco = classificar_risco(autonomia)
```

```
# Armazenamento
```

```
volumes_perdidos.append(perdido)
```

```
volumes_entregues.append(entregue)
```

```
custos_diarios.append(custo)
```

```
autonomias.append(autonomia)
```

```
riscos.append(risco)
```

```
# =====
```

```
# IDENTIFICAÇÃO DO BAIRRO MAIS CRÍTICO
```

```
# =====
```

```
indice_critico = autonomias.index(min(autonomias))
```

```
bairro_critico = bairros[indice_critico]
```

```
# =====
```

```
# TABELA DE RESULTADOS
```

```
# =====
```

```
dados = {
```

```
    "Bairro": bairros,
```

```
    "Demanda (m³)": demandas,
```

```
    "Volume Entregue (m³)": volumes_entregues,
```

```
    "Volume Perdido (m³)": volumes_perdidos,
```

```
    "Custo Diário (R$)": custos_diarios,
```

```
    "Autonomia (dias)": autonomias,
```

```
    "Risco": riscos
```

```
}
```

```
tabela = pd.DataFrame(dados)
```

```
print("\n=====")
```

```
print("RESULTADOS DO SISTEMA")
```

```
print("=====\n")
```

```
print(tabela)
```

```
print("\n=====")
```

```
print(f"Bairro mais crítico: {bairro_critico}")
```

```
print("=====")
```

```
# =====
```

```
# GRÁFICO 1 - DEMANDA X VOLUME ENTREGUE
```

```
# =====
```

```
plt.figure(figsize=(10,6))
```

```
x = range(len(bairros))
```

```
plt.bar(x, demandas, width=0.4, label="Demanda")
```

```
plt.bar(
```

```
    [i + 0.4 for i in x],
```

```
    volumes_entregues,
```

```
    width=0.4,
```

```
    label="Volume Entregue"
```

```
)
```

```
plt.xticks([i + 0.2 for i in x], bairros)
```

```
plt.ylabel("Volume (m³)")
```

```
plt.title("Demanda x Volume Entregue")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# =====
```

```
# GRÁFICO 2 - CUSTO DIÁRIO
```

```
# =====
```

```
plt.figure(figsize=(10,6))
```

```
plt.bar(bairros, custos_diarios)
```

```
plt.ylabel("Custo Diário (R$)")
```

```
plt.title("Custo Diário de Bombeamento")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# =====
```

```
# GRÁFICO 3 - DISTRIBUIÇÃO DOS RISCOS
```

```
# =====
```

```
baixo = riscos.count("Baixo")
```

```
medio = riscos.count("Médio")
```

```
alto = riscos.count("Alto")
```

```
labels = ["Baixo", "Médio", "Alto"]
```

```
valores = [baixo, medio, alto]
```

```
plt.figure(figsize=(7,7))
```

```
plt.pie(
```

```
    valores,
```

```
    labels=labels,
```

```
    autopct='%1.1f%%'
```

```
)
```

```
plt.title("Distribuição dos Níveis de Risco")
```



```
plt.show()
```

```
# =====
```

```
# SIMULAÇÃO DE 7 DIAS
```

```
# =====
```

```
print("\n=====")
```

```
print("SIMULAÇÃO DE 7 DIAS")
```

```
print("=====")
```

```
dias = 7
```

```
volumes_simulados = volumes_iniciais.copy()
```

```
historico = {bairro: [] for bairro in bairros}
```

```
dias_criticos = []
```

```
for dia in range(1, dias + 1):
```

```
    print(f"\n----- DIA {dia} -----")
```

```
    for i in range(len(bairros)):
```

```
# Variação aleatória
```

```
variacao = random.uniform(-0.2, 0.2)
```

```
demanda_dia = demandas[i] * (1 + variacao)
```

```
# Atualização do volume
```

```
volumes_simulados[i] -= demanda_dia
```

```
if volumes_simulados[i] < 0:
```

```
    volumes_simulados[i] = 0
```

```
historico[bairros[i]].append(volumes_simulados[i])
```

```
# Nova autonomia
```

```
autonomia_dia = volumes_simulados[i] / demanda_dia
```

```
risco_dia = classificar_risco(autonomia_dia)
```

```
print(f"\nBairro: {bairros[i]}")
```

```
print(f"Demanda do dia: {demanda_dia:.2f} m3")
```

```
print(f"Volume restante: {volumes_simulados[i]:.2f} m3")
```

```
print(f"Risco: {risco_dia}")
```

```

if risco_dia == "Alto":

    dias_criticos.append(

        f'{bairros[i]} - Dia {dia}'

    )

# =====

# DIAS CRÍTICOS

# =====

print("\n=====")

print("DIAS EM SITUAÇÃO CRÍTICA")

print("=====")

if len(dias_criticos) == 0:

    print("Nenhum bairro entrou em situação crítica.")

else:

    for item in dias_criticos:

        print(item)

```

```
# =====  
# GRÁFICO FINAL - EVOLUÇÃO DOS RESERVATÓRIOS  
# =====
```

```
plt.figure(figsize=(12,6))
```

```
for bairro in bairros:
```

```
    plt.plot(  
        range(1, dias + 1),  
        historico[bairro],  
        marker='o',  
        label=bairro  
    )
```

```
plt.xlabel("Dias")
```

```
plt.ylabel("Volume Disponível (m³)")
```

```
plt.title("Evolução do Volume dos Reservatórios")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
import pandas as pd

import matplotlib.pyplot as plt

import random


# --- FUNÇÕES DE APOIO ---


def calcular_risco(autonomia):

    """Classifica o risco com base na autonomia em dias."""

    if autonomia > 3:

        return "Baixo"

    elif 1 <= autonomia <= 3:

        return "Médio"

    else:

        return "Alto"


def simular_7_dias(bairros_original):

    """Simula o sistema por uma semana com variação de demanda e atualização de estoque."""

    historico_volumes = {b['nome']: [b['volume_atual']] for b in bairros_original}

    eventos_criticos = []


    # Criamos uma cópia para a simulação não afetar os dados da tabela inicial

    bairros_simu = [b.copy() for b in bairros_original]


    for dia in range(1, 8):
```

```

for b in bairros_simu:

    # Demanda varia entre -20% e +20%

    variacao = random.uniform(0.8, 1.2)

    demanda_real = b['demanda'] * variacao


    # Cálculo de perdas e necessidade total

    perda_vol = demanda_real * (b['perda_pct'] / 100)

    total_necessario = demanda_real + perda_vol


    # Consumo do reservatório

    if b['volume_atual'] >= total_necessario:

        b['volume_atual'] -= total_necessario

    else:

        entregue_possivel = b['volume_atual']

        b['volume_atual'] = 0

        eventos_criticos.append(f"DIA {dia}: {b['nome']} ESGOTADO! (Necessário:
{total_necessario:.1f}, Disponível: {entregue_possivel:.1f})")


    # Alerta de criticidade (estoque para menos de 1 dia de demanda original)

    if 0 < b['volume_atual'] < b['demanda']:

        eventos_criticos.append(f"DIA {dia}: {b['nome']} com volume crítico!")


    historico_volumes[b['nome']].append(max(0, b['volume_atual']))


return historico_volumes, eventos_criticos


def gerar_graficos(df, historico_volumes):

```

```
"""Gera visualizações para análise de dados."""
```

```
# Configuração de estilo
```

```
plt.style.use('seaborn-v0_8-muted')
```

```
fig, axs = plt.subplots(2, 2, figsize=(16, 10))
```

```
plt.subplots_adjust(hspace=0.4, wspace=0.3)
```

```
# 1. Comparação Demanda vs Perda
```

```
df.plot(x='Bairro', y=['Demanda (m³)', 'Perda (m³)'], kind='bar', ax=axs[0, 0],  
color=['#3498db', '#e74c3c'])
```

```
axs[0, 0].set_title("Demanda vs Perda na Tubulação", fontweight='bold')
```

```
axs[0, 0].set_ylabel("m³")
```

```
# 2. Custos de Bombeamento
```

```
df.plot(x='Bairro', y='Custo (R$)', kind='bar', ax=axs[0, 1], color='#2ecc71', legend=False)
```

```
axs[0, 1].set_title("Custo Diário de Operação", fontweight='bold')
```

```
axs[0, 1].set_ylabel("Reais (R$)")
```

```
# 3. Evolução do Volume (Linha)
```

```
for bairro, volumes in historico_volumes.items():
```

```
    axs[1, 0].plot(range(8), volumes, marker='o', label=bairro, linewidth=2)
```

```
axs[1, 0].set_title("Evolução dos Reservatórios (7 Dias)", fontweight='bold')
```

```
axs[1, 0].set_xlabel("Dia da Simulação")
```

```
axs[1, 0].set_ylabel("Volume (m³)")
```

```
axs[1, 0].legend()
```

```
axs[1, 0].grid(True, alpha=0.3)
```

```
# 4. Distribuição de Riscos
```

```

cores_risco = {'Baixo': 'green', 'Médio': 'orange', 'Alto': 'red'}

contagem_risco = df['Risco'].value_counts()

contagem_risco.plot(kind='pie', ax=axes[1, 1], autopct='%1.1f%%',
                    colors=[cores_risco.get(x) for x in contagem_risco.index])

axes[1, 1].set_title("Níveis de Risco de Desabastecimento", fontweight='bold')

axes[1, 1].set_ylabel("")

plt.show()

```

```

# --- DADOS DOS 4 BAIRROS ---

```

```

dados_predefinidos = [

    {'nome': 'Centro', 'demanda': 500, 'capacidade': 2000, 'volume_atual': 1800, 'perda_pct':
10, 'custo_m3': 2.50},

    {'nome': 'Vila Nova', 'demanda': 300, 'capacidade': 800, 'volume_atual': 350, 'perda_pct':
15, 'custo_m3': 1.80},

    {'nome': 'Industrial', 'demanda': 1200, 'capacidade': 5000, 'volume_atual': 4500,
'perda_pct': 5, 'custo_m3': 3.20},

    {'nome': 'Jardim Sul', 'demanda': 200, 'capacidade': 500, 'volume_atual': 150, 'perda_pct':
25, 'custo_m3': 1.20}

]

```

```

# --- PROCESSAMENTO PRINCIPAL ---

```

```

print("=== RELATÓRIO TÉCNICO: SISTEMA DE ABASTECIMENTO (4 BAIRROS) ===")

```

```

# Criar DataFrame para cálculos estáticos iniciais

```

```

resultados = []

```

```

for b in dados_predefinidos:

```



```
perda = b['demanda'] * (b['perda_pct'] / 100)

custo = (b['demanda'] + perda) * b['custo_m3']

autonomia = b['volume_atual'] / b['demanda']
```

```
resultados.append({

    'Bairro': b['nome'],

    'Demanda (m³)': b['demanda'],

    'Perda (m³)': round(perda, 2),

    'Custo (R$)': round(custo, 2),

    'Autonomia (Dias)': round(autonomia, 2),

    'Risco': calcular_risco(autonomia)

})
```

```
df_final = pd.DataFrame(resultados)
```

```
# Exibição da Tabela
```

```
print("\n" + df_final.to_string(index=False))
```

```
# Identificação do Ponto Crítico
```

```
bairro_critico = df_final.loc[df_final['Autonomia (Dias)'].idxmin()]
```

```
print(f"\nALERTA MÁXIMO: O bairro '{bairro_critico['Bairro']}' é o mais vulnerável hoje.")
```

```
# Simulação de 7 dias
```

```
hist_vol, alertas = simular_7_dias(dados_predefinidos)
```

```
print("\n--- OCORRÊNCIAS DURANTE A SEMANA (SIMULAÇÃO DE DEMANDA VARIÁVEL) ---")
```

if alertas:

Mostra apenas os 5 primeiros alertas para não poluir

for a in alertas[:8]: print(f"⚠️ {a}")

else:

print("✅ Todos os reservatórios operaram dentro da normalidade.")

Gerar Gráficos

gerar_graficos(df_final, hist_vol)

```
import pandas as pd

import matplotlib.pyplot as plt

import random

# -----

# Funções auxiliares

# -----

def calcular_resultados(bairros, demandas, capacidades, volumes_iniciais, perdas, custos):

    resultados = []

    for i in range(len(bairros)):

        # Volume perdido

        volume_perdido = demandas[i] * (perdas[i] / 100)

        # Volume efetivamente entregue

        volume_entregue = max(0, demandas[i] - volume_perdido)

        # Custo diário

        custo_diario = volume_entregue * custos[i]

        # Autonomia em dias

        autonomia = volumes_iniciais[i] / demandas[i] if demandas[i] > 0 else 0

        # Classificação de risco

        if autonomia > 3:

            risco = "Baixo"

        elif 1 <= autonomia <= 3:
```

```
risco = "Médio"
```

```
else:
```

```
risco = "Alto"
```

```
resultados.append([bairros[i], demandas[i], capacidades[i], volumes_iniciais[i],  
                    volume_entregue, volume_perdido, custo_diario, autonomia, risco])
```

```
return resultados
```

```
def simular_7_dias(bairros, demandas, capacidades, volumes_iniciais, perdas, custos):
```

```
    evolucao = {bairro: [] for bairro in bairros}
```

```
    volumes = volumes_iniciais.copy()
```

```
    dias_criticos = []
```

```
    for dia in range(1, 8):
```

```
        for i in range(len(bairros)):
```

```
            # Variação aleatória da demanda (-20% a +20%)
```

```
            demanda_var = demandas[i] * random.uniform(0.8, 1.2)
```

```
            volume_perdido = demanda_var * (perdas[i] / 100)
```

```
            volume_entregue = max(0, demanda_var - volume_perdido)
```

```
            # Atualiza volume do reservatório
```

```
            volumes[i] = max(0, volumes[i] - volume_entregue)
```

```
            evolucao[bairros[i]].append(volumes[i])
```

```
            # Verifica situação crítica
```

```
            autonomia = volumes[i] / demanda_var if demanda_var > 0 else 0
```

```

    if autonomia < 1:

        dias_criticos.append((dia, bairros[i]))

    return evolucao, dias_criticos

# -----

# Entrada de dados (fixo para 4 bairros)

# -----

bairros = ["Centro", "Bela Vista", "Industrial", "Universitário"]

demandas = [500, 300, 700, 400]      # m³/dia

capacidades = [2000, 1500, 2500, 1800] # m³

volumes_iniciais = [1500, 1200, 2000, 1600] # m³

perdas = [10, 15, 12, 8]             # %

custos = [2.5, 2.0, 3.0, 2.2]        # R$/m³

# -----

# Cálculo inicial

# -----

resultados = calcular_resultados(bairros, demandas, capacidades, volumes_iniciais, perdas,
custos)

# Tabela organizada

df = pd.DataFrame(resultados, columns=["Bairro", "Demanda (m³)", "Capacidade (m³)",
"Volume Inicial (m³)",
"Volume Entregue (m³)", "Volume Perdido (m³)", "Custo Diário (R$)",
"Autonomia (dias)", "Risco"])

print("\nTabela de Resultados:\n")

```

```
print(df)
```

```
# Identificação do bairro mais crítico
```

```
bairro_critico = df.loc[df["Autonomia (dias)"].idxmin(), "Bairro"]
```

```
print(f"\nBairro mais crítico: {bairro_critico}")
```

```
# -----
```

```
# Gráficos
```

```
# -----
```

```
plt.figure(figsize=(10,6))
```

```
plt.bar(df["Bairro"], df["Demanda (m³)"], label="Demanda")
```

```
plt.bar(df["Bairro"], df["Volume Entregue (m³)"], alpha=0.7, label="Volume Entregue")
```

```
plt.title("Comparação Demanda vs Volume Entregue")
```

```
plt.legend()
```

```
plt.show()
```

```
plt.figure(figsize=(10,6))
```

```
plt.bar(df["Bairro"], df["Custo Diário (R$)"], color="orange")
```

```
plt.title("Custo Diário por Bairro")
```

```
plt.show()
```

```
plt.figure(figsize=(8,6))
```

```
df["Risco"].value_counts().plot(kind="pie", autopct="%1.1f%%",  
colors=["green", "yellow", "red"])
```

```
plt.title("Distribuição dos Níveis de Risco")
```

```
plt.ylabel("")
```

```
plt.show()
```

```
# -----
```

```
# Simulação de 7 dias
```

```
# -----
```

```
evolucao, dias_criticos = simular_7_dias(bairros, demandas, capacidades, volumes_iniciais,  
perdas, custos)
```

```
print("\nDias críticos identificados durante a simulação:")
```

```
for dia, bairro in dias_criticos:
```

```
    print(f"Dia {dia}: {bairro} em risco crítico")
```

```
# Gráfico da evolução dos reservatórios
```

```
plt.figure(figsize=(10,6))
```

```
for bairro in bairros:
```

```
    plt.plot(range(1,8), evolucao[bairro], marker="o", label=bairro)
```

```
plt.title("Evolução do Volume dos Reservatórios (7 dias)")
```

```
plt.xlabel("Dia")
```

```
plt.ylabel("Volume disponível (m³)")
```

```
plt.legend()
```

```
plt.show()
```